

# The Functionalizer: Lossless Functional Decomposition for Subword Tokenization

Connor Makowski  
Center for Transportation & Logistics  
Massachusetts Institute of Technology  
Cambridge, MA, USA  
conmak@mit.edu

Willem Guter  
Center for Transportation & Logistics  
Massachusetts Institute of Technology  
Cambridge, MA, USA  
wjguter@mit.edu

July 7, 2026

## Abstract

Standard subword tokenizers either treat every orthographic variation of a word (such as `hello`, `Hello`, `HELLO`, and `Héllö`) as unrelated vocabulary entries, which fragments the embedding space, or discard this variation through lossy normalization. We present the **Functionalizer**, a lossless pre-tokenizer framework that factors orthographic and structural variations into a compositional **opcode/operand** prefix stream before tokenization: a canonical base token (operand) prefixed by parametric transformation operators (opcodes) encoded in the Unicode Private Use Area. We introduce operators covering casing (**CAPITALIZE**), diacritics (13 dedicated opcodes), and character repetition (**REPEAT**, **MULTIREPEAT**), which are fully reversible. Across six natural language and code corpora, the Functionalizer enables complete corpus coverage with significantly smaller vocabularies under unconstrained conditions, reducing actual vocabulary slot requirements by up to **16%**. When looking at sequence lengths, we observe a sharp domain-dependent tradeoff: it compresses indentation-heavy code sequences but inflates natural-language prose sequences. Preliminary downstream evaluations on 25M parameter GPT-2 scale models show that at this scale, the Functionalizer drastically improves code syntax validity and improves code character perplexity while maintaining similar text coherence on prose. These findings demonstrate that functional decomposition can be an effective mechanism for vocabulary-efficient, structurally aware language modeling, and motivate further validation at production scale.

## 1 Introduction

Subword tokenizers face a dilemma. Treating `hello`, `Hello`, and `Héllö` as independent tokens causes **vocabulary expansion**: redundant surface forms consume embedding slots, and a gradient update to `Hello` never benefits `hello`. The alternative, aggressive lowercasing and accent-stripping, is **lossy**. It compresses the vocabulary but permanently discards information the downstream model can never recover.

The Functionalizer takes a third path: **lossless functional decomposition**. Rather than memorizing surface forms or destroying them, it factors variation out into reusable *operators* applied to a single *canonical base*. The design is borrowed directly from Instruction Set Architecture: a CPU does not implement a distinct instruction for every constant (`ADD_1`, `ADD_2`, ...); it separates the operation (opcode) from its data (operand), as in `ADD A, #1`. The Functionalizer applies the same factoring; the base token is the operand, the transformation prefix is the opcode.

### Contributions.

1. A unified opcode/operand framework that handles casing, diacritics, and character repetition under one compositional, parametric, lossless scheme. Prior work addresses these orthographic variations in isolation, either with lossy normalization or with a single operator that does not necessarily reduce tokenization overhead.

2. A concrete Private Use Area (PUA) encoding that is usable in most tokenizers (such as Hugging Face’s BPE) and fully reversible.
3. Empirical validation across six natural language and code corpora, demonstrating that when unconstrained by target vocabulary limits, the Functionalizer reduces the total vocabulary slots required to fully cover a corpus by up to **16%** through the collapse of formatting variations. Regarding sequences, we show a clear domain split: it significantly compresses indentation-heavy code sequences (yielding inference throughput speedups of up to **22.8%**) while introducing a measured sequence length cost (6–9%) on natural-language prose.
4. Downstream language modeling training and evaluations at the  $\sim 25\text{M}$  parameter scale showing that the Functionalizer significantly improves next-token code predictability (a **12.6%** relative improvement in character perplexity for python) and drastically improves basic syntax validity (achieving up to **9.20%** syntax success compared to **0.00%–2.20%** for standard baselines) while matching coherence on natural language prose.

## 2 Related Work

**Subword tokenization.** BPE (Sennrich et al., 2016), WordPiece (Schuster and Nakajima, 2012), and Unigram (Kudo, 2018) build vocabularies bottom-up from frequent merges; byte-level BPE (Radford et al., 2019) avoids out-of-vocabulary failures by operating over 256 byte values. Existing pre-tokenizers that address orthographic variation are either lossy (lowercasing, accent-stripping) or limited to a single operator. For example, Bayram et al. (2025) introduce a Turkish tokenizer that employs a single `<uppercase>` token to fold casing into a shared root embedding. The Functionalizer generalizes these approaches into a fully parametric, multi-operator instruction set covering position, diacritic kind, and repeat count among other potential future extensions.

**Morphology-aware tokenization.** Morfessor (Creutz and Lagus, 2002, 2007) performs unsupervised morpheme segmentation; MorphBPE (Asgari et al., 2025) constrains BPE merges to morpheme boundaries. The Functionalizer is complementary: where morphology-aware methods target linguistic structure, the Functionalizer targets orthographic surface variation, and the two could be composed.

**Tokenization-free models.** ByT5 (Xue et al., 2022), MrT5 (Kallini et al., 2024), and MegaByte (Yu et al., 2023) pursue orthographic robustness by operating on raw bytes or characters, at the cost of much longer sequences. The Functionalizer moves towards similar invariance while retaining subword granularity and avoiding the sequence-length penalty on code.

**Structured Unicode encoding.** SCRIPT-BPE (Land and Arnett, 2025) re-encodes characters by Unicode script and category to remove cross-lingual bias. The normalization literature (e.g., Gorman and Pinter, 2024) documents the downstream cost of inconsistent Unicode handling. The Functionalizer is complementary to both: a structured prefix scheme that can be layered on top of any existing pipeline.

## 3 The Functionalizer Framework

The Functionalizer establishes a parametric, lossless, prefix-based pre-tokenization framework. Instead of tokenizing raw surface forms directly, the framework decomposes orthographic and structural variations into a compositional sequence of non-destructive operators (opcodes) prepended to a canonical base token (operand).

By design, any transformation that is fully reversible (bijective) and can be mapped to character indices can be integrated as an operator within this framework. This includes not only capitalization and combining diacritics, but also structural transformations (e.g., character repetition). Potential future extensions include reversible morphological lemma folding, number/date normalization, prepending of whitespace, and machine learning preprocessing (e.g., correcting likely spelling errors). By isolating the parametric transformation from

the semantic root, the framework enables downstream language models to process clean, shared canonical bases while preserving all orthographic details for lossless reconstruction.

### 3.1 PUA Instruction Layout

Instructions are prepended to the base token as a sequence of PUA codepoints, composed of an operator followed by numeric parameters:

```
[operator] [param_1] [param_2] ... -> [base_token]
```

- **Numeric parameters** (U+E000–U+E0FF): encode integer values 0–255 (`value = codepoint - 0xE000`).
- **Operators** (U+E100–U+EFFF): opcodes consuming a fixed number of parameters. Decoupling opcode from arguments leaves the full plane available for future operators.

### 3.2 Encoding and Reversibility

The transformation pipeline is designed to be fully bijective, ensuring lossless recovery of the original string. For example:

#### Encode:

1. Extract diacritic marks (generating serialization operators) and uppercase positions (generating CAPITALIZE).
2. Strip combining marks and lowercase the remaining characters.
3. Prepend the operator prefix.

#### Decode:

1. Apply operators in reverse order to the base token, restoring diacritics and capitalization.
2. Remove the operator prefix, yielding the original string.

Repeating text can be detected and encoded in the same way. In the decode order, repetition operators execute last: per-piece transforms (CAPITALIZE, diacritics) are applied first. All position parameters (including `pos` for CAPITALIZE/REPEAT, and `start/end` for MULTIREPEAT) refer to character positions **within their individual piece** (after applying any prior operations, but before expansion). The transform is bijective such that the original text is recovered exactly (see Table 6 in the Appendix for worked encoding examples). This fixed decoding sequence introduces a representation constraint: since per-piece transforms are applied first and then repeated, any capitalization or formatting is duplicated across all expanded units (for example, applying CAPITALIZE(0) to base `abc` and repeating it 3 times yields `AbcAbcAbc`, rather than heterogeneously cased units like `Abcabcab`). In such heterogeneous cases, the encoder must fall back to an uncollapsed representation. While this limitation is minor in standard corpora, the framework could in theory be extended to support arbitrary decoding orders (e.g., left-to-right or right-to-left evaluation) to handle complex formatting combinations.

### 3.3 Pipeline Integration

We recommend that the Functionalizer should run on pre-split pieces, not raw text. With the current design, this is because operators address only the first 256 characters (`pos ≤ 255`). This can be extended to cover a larger index space in future iterations. From a more strategic level, this recommendation holds as it can allow a prepended operator/opcode sequence like “capitalize the next character” to be learned as one token during tokenization.

In this implementation’s tests, the custom Split regex isolates each individual space character as a standalone space piece. The Functionalizer’s REPEAT then aggregates consecutive identical space pieces back

into a single opcode and one base space piece (e.g., 12 isolated space pieces  $\rightarrow$  `[REPEAT(0,12)]` + space). Because each space is its own piece, the repetition collapse does not affect character offsets within adjacent word pieces.

By default (`split_operators = true`), operators and parameters are emitted as separate tokens from the canonical base such that each operator and its parameters are tokenized independently. This allows the model to learn operator semantics and parameter distributions separately from the base token embeddings. Other options can be used to emit the entire operator sequence as a single token, which may be beneficial for certain downstream tasks.

During vocabulary training, the BPE merge process is allowed to naturally merge these separate operator and parameter pieces back into combined tokens (such as merging `[CAPITALIZE]` and `[0]` into a single `[CAPITALIZE(0)]` token). In our experiments, BPE training naturally merges high-frequency operator-parameter pairs, which consumes extra vocabulary slots.

## 4 Current Operators and Expected Performance Profiles

### 4.1 Current Operators Specification

The current implementation of the Functionalizer includes operators for capitalization, combining diacritics (detailed in Table 1), and character repetition (detailed in Table 2).

Each capitalization and diacritic variant is mapped to a dedicated 1-parameter operator in the U+E100–U+E10F space (last two reserved), consuming exactly one parameter (`pos`):

Table 1: Capitalization and Diacritic Operators Specification

| Operator / Opcode     | Params           | Action                                                                                        |
|-----------------------|------------------|-----------------------------------------------------------------------------------------------|
| CAPITALIZE (U+E100)   | <code>pos</code> | Uppercase the character at <code>pos</code> .                                                 |
| TILDE (U+E101)        | <code>pos</code> | Apply combining tilde ( <code>\u{0303}</code> ) to the character at <code>pos</code> .        |
| ACUTE (U+E102)        | <code>pos</code> | Apply combining acute ( <code>\u{0301}</code> ) to the character at <code>pos</code> .        |
| GRAVE (U+E103)        | <code>pos</code> | Apply combining grave ( <code>\u{0300}</code> ) to the character at <code>pos</code> .        |
| CIRCUMFLEX (U+E104)   | <code>pos</code> | Apply combining circumflex ( <code>\u{0302}</code> ) to the character at <code>pos</code> .   |
| DIAERESIS (U+E105)    | <code>pos</code> | Apply combining diaeresis ( <code>\u{0308}</code> ) to the character at <code>pos</code> .    |
| MACRON (U+E106)       | <code>pos</code> | Apply combining macron ( <code>\u{0304}</code> ) to the character at <code>pos</code> .       |
| DOT_ABOVE (U+E107)    | <code>pos</code> | Apply combining dot above ( <code>\u{0307}</code> ) to the character at <code>pos</code> .    |
| RING_ABOVE (U+E108)   | <code>pos</code> | Apply combining ring above ( <code>\u{030A}</code> ) to the character at <code>pos</code> .   |
| DOUBLE_ACUTE (U+E109) | <code>pos</code> | Apply combining double acute ( <code>\u{030B}</code> ) to the character at <code>pos</code> . |
| CARON (U+E10A)        | <code>pos</code> | Apply combining caron ( <code>\u{030C}</code> ) to the character at <code>pos</code> .        |
| COMMA_BELOW (U+E10B)  | <code>pos</code> | Apply combining comma below ( <code>\u{0326}</code> ) to the character at <code>pos</code> .  |
| CEDILLA (U+E10C)      | <code>pos</code> | Apply combining cedilla ( <code>\u{0327}</code> ) to the character at <code>pos</code> .      |
| OGONEK (U+E10D)       | <code>pos</code> | Apply combining ogonek ( <code>\u{0328}</code> ) to the character at <code>pos</code> .       |

Repetition operators start at U+E200:

Table 2: Repetition Operators Specification

| Operator / Opcode    | Params                         | Action                                                                                                                   |
|----------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| REPEAT (U+E200)      | <code>pos, count</code>        | Expand the character at <code>pos</code> to <code>count</code> copies.                                                   |
| MULTIREPEAT (U+E201) | <code>start, end, count</code> | Expand the subsequence from <code>start</code> (inclusive) to <code>end</code> (exclusive) to <code>count</code> copies. |

### 4.2 Expected Performance Profiles

We expect performance improvements and training dynamics to manifest in different ways depending on the function of the operator applied. While these effects can be greatly different depending on the operator, we present here two examples we expect to be representative of the current operator categories:

1. **Semantic Invariance and Case Sharing (CAPITALIZE):** Casing variations typically do not alter the core semantic identity of a word (e.g., `hi` vs. `Hi` or `hello` vs. `Hello`). By extracting the capitalization operation to a parametric prefix, casing variants are collapsed into the same base token embedding. The downstream model shares parameter updates across all occurrences, boosting representation sharing and accelerating gradient updates.
2. **Structural Predictability and Compression (REPEAT):** In contrast, repetition operators target structural and syntactic formatting, most notably in programming languages. Repeated spaces or characters (e.g., indentation levels) are represented compactly by a single space character and a parameter value. This avoids fragmenting the sequence or vocabulary with many variants of whitespace blocks (e.g., , , ). Consequently, repeated whitespace is rendered much more understandable and predictable for models, reducing context sequence lengths and enabling cleaner spatial representation of code block structures.

## 5 Experimental Setup

### 5.1 Pipelines and Configurations

To clarify the mechanics of the test configurations, we define the primary components and flags used across our experimental pipelines:

- **Unicode Normalization (NFC):** All tokenization configurations apply Unicode NFC (Normalization Form Canonical Composition) normalization to the raw input text prior to pre-tokenization. NFC standardizes all visual variations of pre-composed characters and decomposed character-combining mark sequences (such as different forms of accented characters) into a single, unified canonical composed character representation (e.g., composing decomposed `e` + `Combining Acute Accent` into the single pre-composed `é`). While not necessary, this can help reduce text variation and improve the efficiency of the Functionalizer’s decomposition.
- **Split Operators (`split_operators`):** A configuration flag that determines whether prepended PUA operators (opcodes) and their numeric arguments (parameters) are split into independent string pieces before BPE subword training. For example, when enabled (`split_operators = true`), BPE receives `[CAPITALIZE(0)]`, and `hello` as separate tokens rather than the fused sequence `[CAPITALIZE(0)]hello`. This prevents the vocabulary from memorizing fused operator-operand structures and allows the model to learn reusable, generalizable embeddings for the operators and arguments. When disabled, the prefix remains attached to the base word.
- **Regex Splitting (`Split`):** Segments raw text into localized character runs (words, numbers, symbols, spaces, or newlines) prior to subword training. This pre-segmentation isolates indentation sequences and keeps character offset counts compact. We compare a custom regex splitter (`\n+|\t+|[ ]+|[\^\\p{L}\p{N}\s_]+`) and the standard Llama 3 pattern (`LLAMA_SPLIT_PAT`).
- **Functionalizer Decompositions:** Governed by three key parameters in the pre-tokenizer:
  - **capitalize:** Identifies uppercase characters, generates a `CAPITALIZE(pos)` opcode, and lowercases the base character.
  - **serialize:** Detects combining diacritical marks, generates diacritic-specific opcodes (e.g., `ACUTE(pos)`), and strips the diacritics from the base character.
  - **repeat:** Identifies runs of 3+ consecutive identical pre-tokenized pieces (or 3+ identical characters within a single piece), collapses them to a single instance, and generates `REPEAT` or `MULTIREPEAT` opcodes. When upstream pre-tokenizers (e.g., the Split regex) isolate each space as its own piece, `repeat` aggregates those split pieces back into one opcode plus one base piece.

For all Functionalizer configurations, the corresponding decoder is configured to mirror the pre-tokenizer flags to ensure losslessness. For downstream language model training (Section 6.2) and downstream inference task evaluation (Section 6.3), we evaluate `Split Only`, `Split + Functionalizer`, and `Split + Functionalizer (Repeat)` configurations.

## 5.2 Datasets, Vocabulary, and Tokenizer Metrics

We conduct experiments across six distinct natural language and source code corpora:

- **Prose:** Wikitext (Salesforce/wikitext-2-raw-v1), TinyStories (roneneldan/TinyStories).
- **Source Code:** Python-Codes (flytech/python-codes-25k) and CodeSearchNet (CSN-Python, CSN-Java, CSN-Go).

We train tokenizers with target vocabulary sizes of **128k**. All of our chosen corpora are exhausted before reaching the 128k target. This means that training continues until no further BPE merges can be performed, representing corpus exhaustion.

To evaluate vocabulary efficiency and compression directly at the tokenizer level, we track two metrics:

1. **Token Compression (Chars/Token):** The average number of visual characters represented per token. A higher ratio indicates stronger text compression and shorter sequence lengths.
2. **Vocab Diff (%):** The percentage reduction in actual vocabulary size achieved by the Functionalizer under unconstrained/exhaustion conditions (`actual_vocab_size < target_vocab_size`), which measures the framework’s ability to cover the same corpus with a smaller vocabulary.

## 5.3 Downstream Training and Evaluation

To assess downstream performance, we train a GPT-2 model (6 layers, 512 embedding dim, 8 attention heads, 16k target vocabulary, ~25M parameters) on each configuration.

- **Training Details:** Trained with a batch size of 16, a learning rate of 5e-4, an AdamW optimizer (weight decay 0.01), a linear learning rate schedule with 200 warmup steps decaying to zero, gradient clipping at 1.0, and a context length of 256 tokens. Training runs for 5,000 steps. Training is repeated across five distinct seeds: 10, 42, 100, 2026, and 7.
- **Inference Downstream Prompts:** Evaluated on 100 validation prompts per dataset. Generation uses greedy decoding up to 50 new tokens (for natural language) or 64 new tokens (for source code).
- **Downstream Evaluation Metrics:**
  - **Per-Character Perplexity (Char PPL):** A normalized measure of language modeling loss to enable direct comparison across tokenizers with differing token lengths. Because tokenizers partition the same text into different numbers of tokens, comparing token-level perplexity is unfair (tokenizers with shorter tokens will show artificially lower perplexity). To normalize, we convert the average token-level cross-entropy loss ( $\mathcal{L}_{\text{token}}$ ) to character-level cross-entropy loss ( $\mathcal{L}_{\text{char}}$ ) using the corpus-level average character-to-token ratio ( $R_{\text{char/token}} = \frac{N_{\text{chars}}}{N_{\text{tokens}}}$ ):

$$\mathcal{L}_{\text{char}} = \frac{\mathcal{L}_{\text{token}}}{R_{\text{char/token}}} \quad (1)$$

The per-character perplexity is then computed as:

$$\text{PPL}_{\text{char}} = \exp(\mathcal{L}_{\text{char}}) = \exp\left(\frac{\mathcal{L}_{\text{token}}}{R_{\text{char/token}}}\right) \quad (2)$$

This ensures that character perplexity represents a fair, tokenizer-independent metric of information density.

- **Throughput (Chars/Sec):** During training, this is computed as the total tokens processed multiplied by the dataset’s average character-to-token ratio (based on raw text length including all spaces and formatting) divided by total training time. During inference, this text speed is calculated on the reconstructed decoded string length (including spaces and newlines, with PUA opcodes decoded back to standard characters) divided by the coherent generation time. To prevent collapsed models from scoring artificially high throughput, this speed is computed exclusively over the sequence prefix generated prior to any repetition collapse (i.e., before any single token repeats > 3 times consecutively).

Table 3: Tokenizer Metrics at 128k Target Vocabulary (Corpus Exhaustion)

| Dataset      | Pipeline Config     | Vocab (Base) | Vocab (Func) | Vocab Diff | Chars/Token (Base) | Chars/Token (Func) |
|--------------|---------------------|--------------|--------------|------------|--------------------|--------------------|
| Wikitext     | Split Only          | 51,780       | 43,700       | −15.60%    | 2.4704             | 2.2659             |
|              | Split Only (Repeat) | 51,780       | 52,013       | +0.45%     | 2.4704             | 2.4750             |
|              | Llama Split         | 53,262       | 45,714       | −14.17%    | 4.2189             | 3.6215             |
| Python-Codes | Split Only          | 33,399       | 28,017       | −16.11%    | 2.1984             | 2.3070             |
|              | Split Only (Repeat) | 33,399       | 33,445       | +0.14%     | 2.1984             | 2.4362             |
|              | Llama Split         | 37,181       | 31,280       | −15.87%    | 3.7522             | 3.2600             |
| TinyStories  | Split Only          | 14,280       | 12,853       | −9.99%     | 2.2937             | 2.1553             |
|              | Split Only (Repeat) | 14,280       | 14,665       | +2.70%     | 2.2937             | 2.2919             |
|              | Llama Split         | 16,981       | 15,639       | −7.90%     | 4.2100             | 3.7533             |
| CSN-Python   | Split Only          | 94,605       | 82,575       | −12.72%    | 1.8928             | 2.5848             |
|              | Split Only (Repeat) | 94,605       | 94,243       | −0.38%     | 1.8928             | 2.7177             |
|              | Llama Split         | 90,834       | 77,785       | −14.37%    | 4.0416             | 3.4345             |
| CSN-Java     | Split Only          | 62,436       | 54,371       | −12.92%    | 2.1356             | 2.3325             |
|              | Split Only (Repeat) | 62,436       | 62,257       | −0.29%     | 2.1356             | 2.6693             |
|              | Llama Split         | 84,810       | 76,664       | −9.60%     | 4.2262             | 3.1723             |
| CSN-Go       | Split Only          | 15,735       | 15,281       | −2.89%     | 1.9750             | 1.8464             |
|              | Split Only (Repeat) | 15,735       | 15,672       | −0.40%     | 1.9750             | 1.9699             |
|              | Llama Split         | 20,458       | 20,125       | −1.63%     | 2.6201             | 2.3779             |

- **Average Tokens Pre-Collapse:** The average number of tokens generated before a single token repeats consecutively more than three times (the threshold for pathological loop collapse). This measures the length of coherent sequence generation before degeneration.
- **Coherence (1 - Repetition):** For prose, the percentage of generated sequences free of pathological n-gram loops. If the model produces an empty or whitespace-only sequence, it is tracked as 0% coherent.
- **Syntax Success Rate:** For source code, the percentage of generations that parse cleanly under Python’s `ast.parse` or Go’s `gofmt`. Empty generations (or whitespace only) are counted as failures.
- **% Empty:** The percentage of generated sequences that are empty or contain only whitespace, metaspace, or raw PUA control characters. While empty sequences are classified as failures for quality metrics (coherence and syntax success), their tokens and generation time are still fully accounted for in the throughput and speed calculations.

## 6 Results

### 6.1 Tokenizer Metrics

To analyze vocabulary usage, token compression, and unconstrained vocabulary requirements (corpus exhaustion), we evaluate configurations across all six datasets. Table 3 presents consolidated metrics at the **128k target vocabulary size** (which represents the unconstrained/corpus exhaustion regime where the tokenizer’s bounds are determined by corpus entropy rather than a hard limit). Each configuration consists of a **Base** version (without decomposition) and a **Functionalizer** version (with decomposition). The configurations labeled as **(Repeat)** (e.g. Split Only (Repeat) or Split + Functionalizer (Repeat)) represent baseline and Functionalizer configurations where only the character repetition operators (**REPEAT**, **MULTIREPEAT**) are active, while capitalization and diacritic serialization operators are disabled.

#### Key Insights and Analysis

**Distinct vocabulary and corpus exhaustion.** At the 128k target scale, all of our datasets saturate before reaching the maximum vocabulary size, leading to corpus exhaustion. Table 3 demonstrates that when unconstrained by a target vocabulary limit, the collapse of formatting variants allows the Functionalizer to achieve complete corpus coverage with a significantly smaller total vocabulary size. The necessary vocabulary size drops by up to **16%** on standard configurations:



- **Wikitext:** Reduces by **15.60%** (43,700 vs. 51,780).
- **Python-Codes:** Reduces by **16.11%** (28,017 vs. 33,399).
- **TinyStories:** Reduces by **9.99%** (12,853 vs. 14,280).
- **CSN-Python:** Reduces by **12.72%** (82,575 vs. 94,605).
- **CSN-Java:** Reduces by **12.92%** (54,371 vs. 62,436).

On **CSN-Go**, the unconstrained vocabulary reduction is minor (**2.89%**), matching the minimal casing and repetition structure targeted by the operators.

**Token compression: the domain split.** The character-to-token ratio (Chars/Token) in Table 3 reveals a sharp division between prose and code.

- **Prose:** On Wikitext and TinyStories, the Functionalizer reduces the Chars/Token ratio (resulting in a **+6.42% to +16.50%** token inflation across configurations, or **+6.42% to +9.02%** on standard splits). This sequence length cost occurs because natural language prose lacks the long repeated sequences that REPEAT collapses, meaning the casing operators represent a net sequence length overhead.
- **Source Code:** On CSN-Python and CSN-Java, the Functionalizer substantially increases the Chars/Token ratio, compressing sequence lengths. This compression saves up to **26.77%** in tokens on CSN-Python and **8.44%** on CSN-Java (increasing up to **30.35%** and **19.99%** respectively when considering repetition collapse only). This is driven by indentation collapse, where long space runs are collapsed into a single character plus a REPEAT operator.

**The Llama Split Code Inflation.** While custom Split configurations compress source code, the **Llama Split** configuration exhibits the opposite behavior, showing sequence length inflation on both CSN-Python (**+17.67%** token inflation, with Chars/Token dropping from 4.0416 to 3.4345) and CSN-Java (**+33.22%** token inflation, with Chars/Token dropping from 4.2262 to 3.1723). This sequence inflation occurs because the Llama split regex does not isolate spaces from adjacent word pieces, which prevents the Functionalizer’s REPEAT operator from collapsing indentation blocks. Consequently, the casing operators introduce a net token overhead without any offsetting spatial compression. This sensitivity highlights that the Functionalizer’s compression effectiveness is tightly coupled to the split regex of the base tokenizer, which we flag as a key area for follow-up research.

**The Go Indentation and CamelCase Exception.** **CSN-Go** acts as the clearest exception to the source code compression rule. As shown in Table 3, the Functionalizer inflates Go sequence lengths by **+6.97%** (reduced Chars/Token). This occurs because:

1. Go’s standard formatter (**gofmt**) enforces tab-based indentation instead of spaces. Tabs are already single-character tokens in BPE, leaving no long space runs for REPEAT to compress.
2. Go’s strict CamelCase conventions for exported functions and identifiers lead to a high frequency of uppercase transitions, which translates to a high frequency of **CAPITALIZE** operators without corresponding compression.

This exception demonstrates that the Functionalizer’s sequence efficiency is highly predictable and depends entirely on the presence of the orthographic structures its operators are designed to target.

## 6.2 Training Dynamics and Language Modeling Performance

We analyze the training behavior and next-token prediction performance of the  $\sim 25$ M parameter GPT-2 model under each tokenizer configuration. Table 4 summarizes the training metrics across TinyStories, CSN-Python, and CSN-Go, averaged over five random seeds.



Table 4: Training Dynamics and Language Modeling Metrics (5 Seeds). Inflation metrics are calculated from unrounded raw token counts.

| Tokenizer Type                  | Vocab Size | Chars/Token | Inflation | Tokens/Sec           | Chars/Sec             | Final Loss          | Token PPL        | Char PPL            |
|---------------------------------|------------|-------------|-----------|----------------------|-----------------------|---------------------|------------------|---------------------|
| <b>Dataset: TinyStories</b>     |            |             |           |                      |                       |                     |                  |                     |
| Split Only                      | 14,280     | 2.295       | 0.00%     | 55498.7 $\pm$ 661.8  | 127354.4 $\pm$ 1518.5 | 1.4980 $\pm$ 0.0048 | 4.47 $\pm$ 0.02  | 1.9209 $\pm$ 0.0040 |
| Split + Functionalizer          | 12,853     | 2.156       | +6.45%    | 53675.9 $\pm$ 1568.6 | 115705.6 $\pm$ 3381.3 | 1.4402 $\pm$ 0.0011 | 4.22 $\pm$ 0.00  | 1.9506 $\pm$ 0.0010 |
| Split + Functionalizer (Repeat) | 14,665     | 2.294       | +0.03%    | 54220.7 $\pm$ 2187.0 | 124387.2 $\pm$ 5017.1 | 1.5125 $\pm$ 0.0023 | 4.54 $\pm$ 0.01  | 1.9335 $\pm$ 0.0020 |
| <b>Dataset: CSN-Python</b>      |            |             |           |                      |                       |                     |                  |                     |
| Split Only                      | 16,000     | 1.901       | 0.00%     | 52237.9 $\pm$ 17.3   | 99306.9 $\pm$ 33.0    | 2.5551 $\pm$ 0.0041 | 12.87 $\pm$ 0.05 | 3.8345 $\pm$ 0.0082 |
| Split + Functionalizer          | 16,000     | 2.620       | -27.44%   | 54584.6 $\pm$ 19.9   | 143018.9 $\pm$ 52.3   | 3.1686 $\pm$ 0.0040 | 23.77 $\pm$ 0.10 | 3.3512 $\pm$ 0.0052 |
| Split + Functionalizer (Repeat) | 16,000     | 2.771       | -31.39%   | 55470.3 $\pm$ 25.3   | 153699.3 $\pm$ 70.0   | 3.3496 $\pm$ 0.0049 | 28.49 $\pm$ 0.14 | 3.3498 $\pm$ 0.0059 |
| <b>Dataset: CSN-Go</b>          |            |             |           |                      |                       |                     |                  |                     |
| Split Only                      | 15,735     | 2.226       | 0.00%     | 60367.7 $\pm$ 23.3   | 134352.3 $\pm$ 51.8   | 2.8526 $\pm$ 0.0093 | 17.33 $\pm$ 0.16 | 3.6029 $\pm$ 0.0151 |
| Split + Functionalizer          | 15,281     | 1.959       | +13.63%   | 60554.2 $\pm$ 36.4   | 118598.1 $\pm$ 71.3   | 2.6738 $\pm$ 0.0066 | 14.50 $\pm$ 0.10 | 3.9166 $\pm$ 0.0133 |
| Split + Functionalizer (Repeat) | 15,672     | 2.224       | +0.07%    | 60627.7 $\pm$ 30.9   | 134833.6 $\pm$ 68.8   | 2.8677 $\pm$ 0.0085 | 17.60 $\pm$ 0.15 | 3.6308 $\pm$ 0.0139 |

## Key Insights and Analysis

**Analyzing Per-Character Perplexity (Char PPL).** Because sequence lengths differ under different tokenizers, Token Perplexity (Token PPL) is not directly comparable. We therefore compute the Per-Character Perplexity (Char PPL) as a normalized measure of language modeling performance.

- **CSN-Python (Source Code):** The Functionalizer configurations consistently and significantly outperform their baseline counterparts. For instance, the **Split + Functionalizer** achieves a Char PPL of **3.3512** compared to the **Split Only** baseline at **3.8345** (a relative improvement of **12.6%**). This indicates that collapsing space runs into simple structural parameters makes code indentation structures highly predictable, boosting the model’s overall representational capability.
- **CSN-Go (Source Code):** Go represents the counter-case. Due to tab indentation (which already behaves as clean single-character tokens in BPE) and capital-dense CamelCase exports, the Functionalizer inflates Go sequence lengths without substantial offsetting compression (causing a +13.63% sequence inflation). Consequently, Char PPL degrades under the Go configuration (e.g., **3.9166** for Split + Functionalizer vs. **3.6029** for Split Only).
- **TinyStories (Prose):** Consistent with the sequence inflation on prose (+6.45%), the Split + Functionalizer experiences a minor Char PPL degradation (1.9506 vs. 1.9209).

**Computational Throughput.** The Functionalizer can improve character level throughput for certain corpora.

- **Throughput (Chars/Sec):** The Functionalizer significantly increases content throughput (Chars/Sec) on corpora where it reduces sequence length. For example, on CSN-Python, the Split + Functionalizer generates **143,018.9 chars/sec** compared to **99,306.9 chars/sec** for Split Only. This represents a **44.0%** increase in content throughput. This gain is driven primarily by the higher chars/token ratio achieved through sequence compression: raw token generation rates (Tokens/Sec) remain comparable across all configurations ( $\sim$ 52,000–62,000 tokens/sec), so more characters are delivered per generated token rather than more tokens per second. On prose (TinyStories), character throughput slightly decreases (**115,705.6** vs. **127,354.4 chars/sec**), in line with the sequence length cost.

## 6.3 Downstream Inference and Downstream Tasks

We perform greedy decoding evaluations on validation prompts to assess inference latency and text generation success. Table 5 lists the results (Part a for prose, Part b for code).

## Key Insights and Analysis

**Downstream Code Generation Syntax Success Rate.** One of our most striking results is the impact of decomposition on source code syntax validity under highly limited model size. At the 25M parameter GPT-2 scale, models struggle to learn basic syntax and indentation, as evidenced by the zero success rates of standard tokenizers on CSN-Python (e.g., **0.00%** for Split Only).

Table 5: Downstream Inference Metrics (100 Prompts, 5 Seeds)

## (a) Prose Generation Metrics

| Tokenizer Type                                 | Avg Tokens | Avg Tokens Pre-Collapse | Avg Chars  | Tokens / Sec | Chars / Sec (Text Speed) | % Empty    | Coherence (1 – Rep.) (%) |
|------------------------------------------------|------------|-------------------------|------------|--------------|--------------------------|------------|--------------------------|
| <b>Dataset: TinyStories (Prose Generation)</b> |            |                         |            |              |                          |            |                          |
| Split Only                                     | 7.6 ± 0.5  | 3.6 ± 0.5               | 17.4 ± 1.6 | 303.0 ± 16.0 | 1117.9 ± 42.8            | 1.0 ± 0.0% | 99.00 ± 0.00%            |
| Split + Functionalizer                         | 15.7 ± 1.1 | 11.7 ± 1.1              | 40.1 ± 4.0 | 311.0 ± 3.3  | 954.5 ± 28.2             | 2.2 ± 1.2% | 97.80 ± 1.17%            |
| Split + Functionalizer (Repeat)                | 7.4 ± 0.3  | 3.4 ± 0.3               | 16.6 ± 0.9 | 307.2 ± 6.1  | 1134.5 ± 25.2            | 1.0 ± 0.0% | 99.00 ± 0.00%            |

## (b) Source Code Generation Metrics

| Tokenizer Type                               | Avg Tokens | Avg Tokens Pre-Collapse | Avg Chars   | Tokens / Sec | Chars / Sec (Text Speed) | % Empty      | Syntax Success Rate (%) |
|----------------------------------------------|------------|-------------------------|-------------|--------------|--------------------------|--------------|-------------------------|
| <b>Dataset: CSN-Python (Code Generation)</b> |            |                         |             |              |                          |              |                         |
| Split Only                                   | 6.0 ± 1.3  | 2.0 ± 1.3               | 7.9 ± 2.4   | 313.3 ± 6.0  | 626.7 ± 79.1             | 52.0 ± 25.7% | 0.00 ± 0.00%            |
| Split + Functionalizer                       | 9.1 ± 1.1  | 5.1 ± 1.1               | 15.6 ± 3.1  | 316.8 ± 2.9  | 507.6 ± 200.1            | 77.6 ± 11.6% | <b>9.20 ± 9.00%</b>     |
| Split + Functionalizer (Repeat)              | 7.3 ± 0.2  | 3.3 ± 0.2               | 12.1 ± 0.6  | 317.5 ± 4.1  | 769.8 ± 25.2             | 92.6 ± 2.9%  | 0.00 ± 0.00%            |
| <b>Dataset: CSN-Go (Code Generation)</b>     |            |                         |             |              |                          |              |                         |
| Split Only                                   | 8.5 ± 3.7  | 4.6 ± 3.8               | 10.6 ± 6.1  | 314.6 ± 3.3  | 401.1 ± 74.1             | 73.2 ± 23.7% | 2.20 ± 3.12%            |
| Split + Functionalizer                       | 10.7 ± 7.6 | 7.0 ± 8.2               | 13.6 ± 11.3 | 314.6 ± 1.9  | 431.5 ± 28.4             | 71.8 ± 26.3% | <b>4.00 ± 4.65%</b>     |
| Split + Functionalizer (Repeat)              | 9.9 ± 3.9  | 6.1 ± 4.0               | 13.4 ± 7.4  | 312.2 ± 4.7  | 436.9 ± 81.8             | 67.6 ± 21.7% | 2.80 ± 4.17%            |

By contrast, integrating the Functionalizer pre-tokenizer drastically improves the model’s capacity to output valid code syntax at this scale. The **Split + Functionalizer** configuration reaches a syntax success rate of **9.20%** (compared to **0.00%** for the Split Only baseline). The custom **Split** regex preserves newlines (`\n`) and tabs (`\t`) losslessly, allowing the model to process multi-line sequences. In this setup, the standard Split pre-tokenizer splits the space character from adjacent words, which allows the REPEAT operator to fully aggregate the run of consecutive spaces (e.g., 4 spaces collapse to `\ue200\ue000\ue004` and a space). We note that these syntax success rates carry substantial variance across seeds (e.g., **9.20 ± 9.00%**), reflecting sensitivity to weight initialization at this model scale; the consistent direction of improvement of the Functionalizer configuration on CSN-Python, however, supports the pattern as a real effect rather than noise. This indicates that by offloading the cognitive load from vocabulary and structural whitespace into deterministic, parameter-based instructions, small language models can generate syntactically sound structures that compile/parse. This does not mean that the results are production-ready, but it does suggest that the Functionalizer can help small models learn to represent and generate structured content more effectively.

**Coherent Generation Length (Avg Tokens Pre-Collapse).** A critical challenge in low-parameter language modeling is the tendency to fall into early pathological looping or generation collapse (e.g., repeating spaces, newlines, or specific phrases infinitely). In Table 5, we track the average number of tokens generated before a single token repeats consecutively more than three times (*Avg Tokens Pre-Collapse*). We find that models trained with the Functionalizer generate substantially longer sequences before collapsing. For example, on the **TinyStories** prose dataset, the Split + Functionalizer model generates an average of **11.7** tokens pre-collapse compared to only **3.6** tokens for the Split Only baseline. Similarly, on **CSN-Python**, the Functionalizer increases pre-collapse tokens from **2.0** to **5.1**, and on **CSN-Go** from **4.6** to **7.0**. When paired with similar coherence levels on prose (e.g., 97.80% vs. 99.00%) and significantly higher syntax success rates on code, these metrics indicate that the Functionalizer allows the model to produce longer content without degenerating. While we avoid making definitive causal claims, this behavior suggests that by offloading surface variations, the model can maintain a more stable context representation over longer generation sequences, which may correlate with better representation learning or training dynamics.

**Downstream Latency and Generation Speed.** During inference generation (which is autoregressive and token-by-token), we observe mixed results in generation speed in terms of actual content (characters generated per second). On CSN-Go, the Split + Functionalizer configuration achieves a text speed of **431.5 chars/sec** compared to **401.1 chars/sec** for Split Only (a **7.6%** speedup). For configurations targeting repetition only (Split + Functionalizer (Repeat)), we observe more consistent improvements: on CSN-Python, it generates **769.8 chars/sec** compared to **626.7 chars/sec** for Split Only (a **22.8%** speedup), and on TinyStories it generates **1134.5 chars/sec** compared to **1117.9 chars/sec** (a **1.5%** speedup). However, the full Split + Functionalizer configuration is slower on TinyStories (**954.5 chars/sec**) and CSN-Python (**507.6 chars/sec**) compared to the Split Only baseline, reflecting the sequence length overhead of casing operators when they are not offset by structural compression. We note that the raw generation speeds during inference

are orders of magnitude lower than the training throughput metrics reported in Table 4. This discrepancy is expected and stems from the difference in computation modes.

## 7 Discussion

The tokenizer-level results support a specific, bounded claim: **the Functionalizer is a vocabulary-efficiency and code-compression tool, not a universal improvement**. It eliminates casing-induced vocabulary fragmentation and shortens code sequences substantially (token usage), at the price of longer prose sequences. For a code-domain tokenizer, this tradeoff is clearly favorable; for prose-dominant workloads, the value depends on downstream gains not yet demonstrated at production scale.

**The Serving Cost and Sequence Length Trade-off.** A critical open question is the trade-off between vocabulary size reduction and sequence length inflation. Under modern LLM deployment paradigms, inference is frequently bottlenecked by memory bandwidth, where serving costs and latencies scale with context sequence length (driven by KV cache size and attention computation overhead). In this light, a **6–9%** sequence length inflation on prose represents a non-trivial penalty. Conversely, a **9–16%** (excepting Go) reduction in vocabulary size reduces the model’s embedding parameters, which can decrease peak memory footprint and training requirements. Characterizing the exact boundary where vocabulary slot savings outweigh sequence length inflation, under varying hardware constraints (compute-bound training vs. memory-bandwidth-bound inference), remains an important open question for future research.

The preliminary inference results in Section 6.3 are consistent with the vocabulary compaction story. The Functionalizer matches Split Only coherence on prose while achieving higher vocabulary efficiency. With that said, a 25M parameter model at short generation lengths is insufficient to validate or invalidate any production scale hypothesis. The open questions are squarely enumerated in Subsection 7.1.

It is also worth noting that at this small model scale (25M parameters) and short context length (maximum 50–64 generated tokens), the computation time is not the primary bottleneck during inference. Host-device synchronization (e.g., retrieving the token ID via blocking CPU-GPU calls) and CUDA kernel launch overhead dominate the latency. Consequently, hardware-level optimizations like Key-Value (KV) caching yield negligible differences in generation throughput at this scale. The reported downstream speeds reflect these systemic overhead constraints, and we expect the relative generation speedups of Functionalizer configurations to widen as model parameters and generation sequence lengths scale up.

Additionally, we expect that as more operators are implemented within the Functionalizer framework, they could further improve performance, especially when training models on specific tasks.

### 7.1 Limitations

1. **Downstream evaluation is preliminary.** Inference results are at  $\sim 25$ M parameters (Section 6.3). The vocabulary compression  $\rightarrow$  performance link is not validated at production scale, where we expect gains may shift to training and inference efficiency rather than test performance.
2. **Prose costs sequence length.** Up to +9% more tokens on Wikitext at 128k. For prose-dominant workloads this is a net cost unless offset by downstream gains not yet shown.
3. **256-character position limit.** Transforms beyond index 255 are silently dropped; runs of  $\geq 256$  identical characters are not collapsed. Rare in practice, but a hard boundary under adversarial or unusual inputs. Bumping the starting codepoints of the operators to higher PUA sub-blocks (e.g., shifts starting at U+EA00 and U+EB00) would expand the numeric parameter range and mitigate this constraint, though we do not focus on this extension in the current work.
4. **Fixed diacritic coverage.** The serialization framework supports 13 combining marks. Scripts and marks outside this set fall back to non-functionalizer behavior.
5. **Pareto frontier uncharacterised.** We measure vocabulary size and token usage independently; we do not characterise the joint tradeoff or its interaction with model size.
6. **Incomplete operator ablations.** Beyond some tests with repetition only, we did not isolate the individual contribution of each operator (e.g., casing vs. repetition) in our downstream evaluations.

## 7.2 Future Work

While our preliminary evaluations establish the feasibility and efficiency of functional decomposition, several promising directions remain to expand the framework:

- **Production-Scale Validation:** The most critical next step is extending downstream training and inference validation to standard LLM scales (1B+ parameters). This will evaluate whether the vocabulary compression gains translate directly to training throughput, data efficiency, and downstream task accuracy (especially on casing-sensitive tasks like Named Entity Recognition).
- **Richer Operator Sets:** Expanding and validating advanced bijective operators, including:
  - *Morphological Lemma Folding:* Factoring morphological inflection markers (e.g., [PLURAL], [PAST\_TENSE]) from canonical base lemmas to compress morphologically rich languages.
  - *Structured Pattern Folding:* Factoring structured data runs (e.g., dates, IP addresses, hashes) into parameterized instructions over base operand tokens.
  - *Lossless Spelling Normalization:* Factoring minor typographical errors into parameterized edit operators applied to shared base roots.

## 8 Conclusion

The Functionalizer factors orthographic and structural variation out of the subword tokenizer vocabulary into a compositional, parametric opcode/operand prefix stream encoded in the Unicode Private Use Area. By isolating surface variations from semantic roots, the framework achieves complete corpus coverage with much smaller vocabularies under unconstrained conditions, reducing actual vocabulary slot requirements by up to **16%** (averaging **11.7%** fewer slots overall across split only vs split + functionalizer configurations and datasets).

Downstream training and evaluation on 25M parameter language models reveal a domain-dependent tradeoff: on indentation-dense code, the Functionalizer delivers substantial sequence-length compression, improves next-token predictability, and drastically improves code generation syntax validity (from near-zero for standard baselines up to **9.20%**). On natural-language prose, the casing operators introduce a minor sequence length cost (+6% to +9%) which degrades generation speed (showing a 14.6% slowdown), though the repetition-only configuration (Split + Functionalizer (Repeat)) maintains a minor 1.5% speedup on TinyStories (while matching baseline coherence). On CSN-Python code generation, the repetition-only configuration yields a substantial speedup of 22.8% (at the cost of a higher empty generation rate).

While production-scale validation remains an open question, these preliminary results demonstrate that factoring orthographic complexity out of tokenizers is a viable path toward vocabulary-efficient, structurally aware language modeling.

## A Worked Encoding Examples

Table 6: Worked Encoding Examples

| Input      | Operators              | Base  | Encoded prefix                   |
|------------|------------------------|-------|----------------------------------|
| Hello      | CAPITALIZE(0)          | hello | \u{E100}\u{E000}                 |
| Héllö      | ACUTE(1) CAPITALIZE(0) | hello | \u{E102}\u{E001}\u{E100}\u{E000} |
| #####      | REPEAT(0, 6)           | #     | \u{E200}\u{E000}\u{E006}         |
| (4 spaces) | REPEAT(0, 4)           |       | \u{E200}\u{E000}\u{E004}         |
| abcbabcabc | MULTIREPEAT(0, 3, 3)   | abc   | \u{E201}\u{E000}\u{E003}\u{E003} |

## Data and Code Availability

All tokenizer configurations, training scripts, evaluation pipelines, dataset acquisition scripts, and result files are publicly available at <https://github.com/connor-makowski/functionalizer>. A working implementation is available at <https://github.com/connor-makowski/tokenizers> on the functionalizer branch.

## References

- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *arXiv preprint arXiv:1508.07909*.
- Kudo, T. (2018). Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. *arXiv preprint arXiv:1804.10959*.
- Schuster, M., & Nakajima, K. (2012). Japanese and Korean voice search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 5149–5152). IEEE.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Blog*.
- Creutz, M., & Lagus, K. (2002). Unsupervised Discovery of Morphemes. In *Proceedings of the ACL-02 Workshop on Morphological and Phonological Learning*.
- Creutz, M., & Lagus, K. (2007). Unsupervised Models for Morpheme Segmentation and Morphology Learning. *ACM Transactions on Speech and Language Processing (TSLP)*, 4(1), 3-es.
- Asgari, E., El Kheir, M., & Sadraei Javaheri, A. (2025). MorphBPE: A Morpho-Aware Tokenizer Bridging Linguistic Complexity for Efficient LLM Training Across Morphologies. *arXiv preprint arXiv:2502.00894*.
- Bayram, M. A., Fincan, A. A., Gümüş, A. S., Karakaş, S., Diri, B., Yıldırım, S., & Çelik, D. (2025). Tokens with Meaning: A Hybrid Tokenization Approach for Turkish. *arXiv preprint arXiv:2508.14292*.
- Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., & Raffel, C. (2022). ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models. *arXiv preprint arXiv:2105.13626*.
- Kallini, J., Murty, S., Manning, C. D., Potts, C., & Csordás, R. (2024). MrT5: Dynamic Token Merging for Efficient Byte-level Language Models. *arXiv preprint arXiv:2410.20771*.
- Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., & Lewis, M. (2023). MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers. *arXiv preprint arXiv:2305.07185*.
- Land, S., & Arnett, C. (2025). BPE Stays on SCRIPT: Structured Encoding for Robust Multilingual Pretokenization. *arXiv preprint arXiv:2505.24689*.
- Gorman, K., & Pinter, Y. (2024). Don’t Touch My Diacritics. *arXiv preprint arXiv:2410.24140*.